

## Module 3

### Inheritance:

As the name suggests, inheritance means to take something that is already made. It is one of the most important features of Object Oriented Programming. It is the concept that is used for **reusability** purpose.

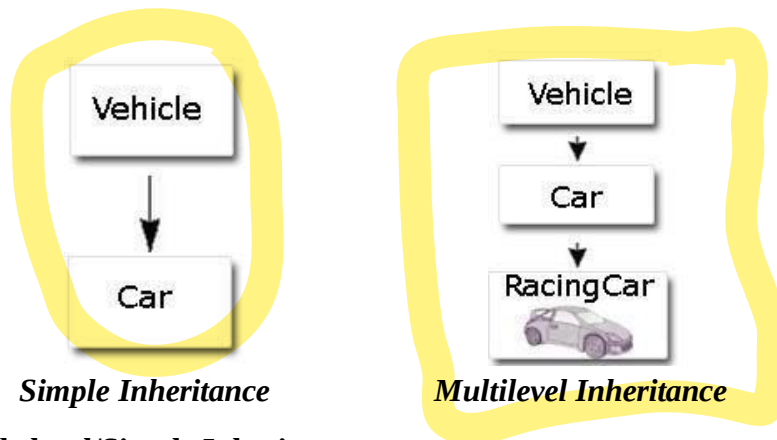
Inheritance is the mechanism through which we can derive classes from other classes.

The derived class is called as child class or the subclass or we can say the extended class and the class from which we are deriving the subclass is called the base class or the parent class.

To derive a class in java the keyword **extends** is used. The following kinds of inheritance are there in java.

***Types of Inheritance 1. Single level/Simple Inheritance 2. Multilevel Inheritance 3. Multiple Inheritance (Java doesn't support Multiple inheritance but we can achieve this through the concept of Interface.)***

### **Pictorial Representation of Simple and Multilevel Inheritance**



### Single level/Simple Inheritance

When a subclass is derived simply from its parent class then this mechanism is known as simple inheritance. In case of simple inheritance there is only a sub class and its parent class. It is also called single inheritance or one level inheritance.

### Example

```
class A
{
    int x;        int y;
    int get(int p, int q)
    {
        x=p;
        y=q;
        return(0);
    }
    void Show()
    {
        System.out.println(x);
    }
}
class B extends A
{
    public static void main(String args[])
    {
        A a = new A();
        a.get(5,6);
        a.Show();
    }
}
```

```
void display()
{
    System.out.println("y");    //inherited "y" from class A
}
}
```

The syntax for creating a subclass is simple. At the beginning of your class declaration, use the extends keyword, followed by the name of the class to inherit from:

```
class A
{
    }  class B extends A           //B is a subclass of super
class A.
{
}
}
```

### ***Multilevel Inheritance***

When a subclass is derived from a derived class then this mechanism is known as the multilevel inheritance.

The derived class is called the subclass or child class for its parent class and this parent class works as the child class for its just above ( parent ) class.

Multilevel inheritance can go up to any number of level.

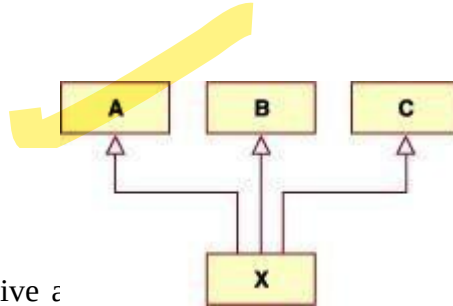
```
class A
{
    int x;
    int y;
    int get(int
p, int q)
    {
        x=p;
        y=q;
        return(0);
    }
    void Show()
    {
        System.out.println(x);
    }
}

class B extends A
{
    void
Showb()
    {
        System.out.println("B");
    }
}

class C extends B
{
    void display()
    {
        System.out.println("C");
    }
    public static void
main(String args[])
    {
        A a = new A();
        a.get(5,6);
        a.Show();
    }
}
OUTPUT
5
```

### Multiple Inheritance

The mechanism of inheriting the features of more than one base class into a single class is known as multiple inheritance. Java does not support multiple inheritance but the multiple inheritance can be achieved by using the interface.



Here you can derive a class from more than one direct base class. Deriving a class from more than one direct base class is called multiple inheritance.

### Java does not support multiple Inheritance

In Java Multiple Inheritance can be achieved through use of Interfaces by implementing more than one interfaces in a class.

### Super keyword

The super is java keyword. As the name suggest super is used to access the members of the super class. It is used for two purposes in java.

The first use of keyword super is to access the hidden data variables of the super class hidden by the sub class.

Example: Suppose class A is the super class that has two instance variables as int a and float b. class B is the subclass that also contains its own data members named a and b. then we can access the super class (class A) variables a and b inside the subclass class B just by calling the following command. **super.member;**

Here member can either be an instance variable or a method. This form of super most useful to handle situations where the local members of a subclass hides the members of a super class having the same name. The following example clarifies all the confusions.

Example:

```

class
A
{
    int    a;
    float b;    void
Show()
{
    System.out.println("b in super class: " + b);
}
}
class B extends A
{
    int a;
    float b;
    B( int p, float q)
    {
        a = p;
        super.b = q;
    }
    void Show()
    {
        super.Show();
        System.out.println("b in super class: " + super.b);
        System.out.println("a in sub class:    " + a);
    }
}

class Mypgm
{
    public static void main(String[] args)
    {
        B subobj = new B(1, 5);
        subobj.Show();
    }
}

```

OUTPU

T

b in super class: 5.0 b

in super class: 5.0 a in

sub class: 1

**Use of super to call super class constructor:** The second use of the keyword super in java is to call super class constructor in the subclass. This functionality can be achieved just by using the following command.

`super(param-list);`

Here parameter list is the list of the parameter requires by the constructor in the super class. super must be the first statement executed inside a super class constructor. If we want to call the default constructor then we pass the empty parameter list. The following program illustrates the use of the super keyword to call a super class constructor.

**Example:**

```
class A
{
    int a;
    int b;
    int c;
    A(int p, int q, int r)
    {
        a=p;
        b=q;
        c=r;
    }
}
class B extends A
{
    int d;
    B(int l, int m, int n, int o)
    {
        super(l,m,n); d=o;
    }
    void Show()
```

```

        {
            System.out.println("a = " + a);
            System.out.println("b = " + b);
            System.out.println("c = " + c);
            System.out.println("d = " + d);
        }
    }
}
class Mypgm
{
    public static void main(String
args[])
    {
        B b = new B(4,3,8,7);

                                b.Show();
    }
}
OUTPUT
a = 4 b
= 3 c =
8 d =
7

```

## Method Overriding

Method overriding in java means a subclass method overriding a super class method.

Superclass method should be non-static. Subclass uses extends keyword to extend the super class. In the example **class B** is the sub class and **class A** is the super class. **In overriding methods of both subclass and superclass possess same signatures.**

Overriding is used in modifying the methods of the super class. In overriding return types and constructor parameters of methods should match.

Below example illustrates method overriding in java.

Example:



```
class A
{
    int i;
    A(int a, int b)
    {
        i = a+b;
    }
    void
    add()
    {   System.out.println("Sum of a and b is: " + i);
    }
}
class B extends A
{
    int j;
    B(int a, int b, int c)
    {
        super(a, b); j
        = a+b+c;
    }
    void
    add()
    {
        super.add();
        System.out.println("Sum of a, b and c is: " + j);
    }
}
class MethodOverriding
{
    public static void main(String
    args[])
    {
        B b = new B(10, 20, 30);
        b.ad
        d();
    }
}
```

**OUTPUT**

Sum of a and b is: 30 Sum  
of a, b and c is: 60

## **Method Overloading**

Two or more methods have the same names but different argument lists. The arguments may differ in type or number, or both. However, the return types of overloaded methods can be the same or different is called **method overloading**.

An example of the method overloading is given below:

**Example:**

```
class MethodOverloading
{
    int add( int a,int b)
    {
        return(a+b);
    }
    float add(float a,float b)
    {
        return(a+b);
    }
    double add( int a, double b,double c)
    {
        return(a+b+c);
    }
}
class MainClass
{
    public static void main( String arr[] )
    {
        MethodOverloading mobj = new MethodOverloading ();
        System.out.println(mobj.add(50,60));
        System.out.println(mobj.add(3.5f,2.5f));
        System.out.println(mobj.add(10,30.5,10.5));
    }
}
OUTPUT
110
6.0
51.0
```

## Abstract Class

**abstract keyword** is used to make a class abstract.

Abstract class can't be instantiated with new operator.

**We can use abstract keyword to create an abstract method; an abstract method doesn't have body.**

If classes have abstract methods, then the class also needs to be made abstract using abstract keyword, else it will not compile.

Abstract classes are used to provide common method implementation to all the subclasses or to provide default implementation. **Example Program:**

```
abstract Class AreaPgm
{
    double dim1,dim2;
    AreaPgm(double x, double y)
    {
        dim1=x
        ;
        dim2=y
        ;
    }
    abstract double area();
}
class rectangle extends AreaPgm
{
    rectangle(double a,double b)
    {
        super(a,b);
    }
    double area()
    {
        System.out.println("Rectangle Area");
        return dim1*dim2;
    }
}
class triangle extends figure
{
    triangle(double x,double y)
    {
        super(x,y);
    }
    double area()
    {
        System.out.println("Traingle Area");
        return dim1*dim2/2;
    }
}
class MyPgm
{
```

```

public static void main(String args[])
{

    AreaPgm a=new AreaPgm(10,10); // error, AreaPgm is a abstract class.

    rectangle r=new rectangle(10,5);
    System.out.println("Area="+r.area());

    triangle t=new triangle(10,8);
    AreaPgm      ar;
    ar=obj; System.out.println("Area="+ar.area());

}
}

```

### final Keyword In Java

The **final keyword** in java is used to restrict the user. The final keyword can be used in many context. Final can be:

1. variable
2. method
3. class

**1) final variable:** If you make any variable as final, you cannot change the value of final variable(It will be constant).

**Example:** There is a final variable speedlimit, we are going to change the value of this variable, but It can't be changed because final variable once assigned a value can never be changed.

```

class Bike
{
    final int speedlimit=90;//final variable
    void run()
    {
        speedlimit=400;
    }
}
Class MyPgm

```

```
{ public static void main(String args[])
{
    Bike obj=new Bike();
    obj.run();
}
}
```

Output: Compile Time Error

---

**2) final method:** If you make any method as final, you cannot override it.

**Example:** class Bike

```
{    final void run()
{
    System.out.println("running");
}
} class Honda extends
Bike
{ void run()
{
    System.out.println("running safely with 100kmph");
} }
```

Class MyPgm

```
{    public static void main(String args[])
{
    Honda honda= new Honda();  honda.run();
}
}
```

Output: Compile Time Error

**3) final class:** If you make any class as final, you cannot extend it.

**Example:**

```
final class Bike
{
}
```

class Honda extends Bike

```
{
    void run()
}
```

```
        {  
            System.out.println("running safely with 50kmph");  
        }  
    }
```

Class MyPgm

```
{  
  
    public static void main(String args[])  
    {  
        Honda honda= new Honda();  
        honda.run();  
    }  
}
```

**Output:Compile Time Error**

## **MODULE 4:**

### **Packages and Interfaces:**

#### **Packages:**

- The name of each example class was taken from the same name space. This means that a unique name had to be used for each class to avoid name collisions.
- **Java provides a mechanism for partitioning the class name space into more manageable chunks. This mechanism is the package.**
- The package is both a naming and a visibility control mechanism. You can define classes inside a package that are not accessible by code outside that package. You can also define class members that are only exposed to other members of the same package.

#### **Defining a Package:**

- To create a package is quite easy: simply include a **package** command as the first statement in a Java source file.
- Any classes declared within that file will belong to the specified package.
- The **package** statement defines a name space in which classes are stored. If you omit the **package** statement, the class names are put into the default package, which has no name.

## **package *pkg*;**

Here, *pkg* is the name of the package.

For example, the following statement creates a package called **MyPackage**.

```
package MyPackage;
```

- Java uses file system directories to store packages. For example, the **.class** files for any classes you declare to be part of **MyPackage** must be stored in a directory called **MyPackage**.
- Remember that case is significant, and the directory name must match the package name exactly.
- More than one file can include the same **package** statement. The **package** statement simply specifies to which package the classes defined in a file belong.



- You can create a hierarchy of packages. To do so, simply separate each package name from the one above it by use of a period.

The general form of a multileveled package statement is shown here:

**package *pkg1*[.*pkg2*[.*pkg3*]];**

- A package hierarchy must be reflected in the file system of your Java development system.
- For example, a package declared as

**package java.awt.image;**

### Finding Packages and CLASSPATH:

- Packages are mirrored by directories.
- The Java run-time system know where to look for packages that you create
  - ➔ First, by default, the Java run-time system uses the current working directory as its starting point. Thus, if your package is in a subdirectory of the current directory, it will be found.
  - ➔ Second, you can specify a directory path or paths by setting the **CLASSPATH** environmental variable.
  - ➔ Third, you can use the **-classpath** option with **java** and **javac** to specify the path to your classes.
- For example, consider the following package specification:

**package MyPack**

can be executed from a directory immediately above **MyPack**, or the **CLASSPATH** must be set to include the path to **MyPack**, or the **-classpath** option must specify the path to **MyPack** when the program is run via **java**.

- When the second two options are used, the class path *must not* include **MyPack**, itself.
- It must simply specify the *path to MyPack*. For example, in a Windows environment, if the path to **MyPack** is

**C:\MyPrograms\Java\MyPack**

- Then the class path to **MyPack** is

**A Short Package Example:**

```
package MyPack;

class Balance {
    String name;
    double bal;
    Balance(String n, double b) {
        name = n;
        bal = b;
    }
    void show() {
        if(bal<0)
            System.out.print("--> ");
        System.out.println(name + ": $" + bal);
    }
}

class AccountBalance {
    public static void main(String args[]) {
        Balance current[] = new Balance[3];
        current[1] = new Balance("Will Tell", 157.02);
        current[2] = new Balance("Tom Jackson", -12.33);
        for(int i=0; i<3; i++) current[i].show();
    }
}
```

- Call this file **AccountBalance.java** and put it in a directory called **MyPack**.
- Next, compile the file. Make sure that the resulting **.class** file is also in the **MyPack** directory.
- Then, try executing the **AccountBalance** class, using the following command line:

**java MyPack.AccountBalance**

**Access Protection:**

- Packages add another dimension to access control.

- Java provides many levels of protection to allow fine-grained control over the visibility of variables and methods within classes, subclasses, and packages. Classes and packages are both means of encapsulating and containing the name space and scope of variables and methods.
- Packages act as containers for classes and other subordinate packages.
- Classes act as containers for data and code. The class is Java's smallest unit of abstraction.
- Because of the interplay between classes and packages, Java addresses four categories of visibility for class members:
  - Subclasses in the same package
  - Non-subclasses in the same package
  - Subclasses in different packages
  - Classes that are neither in the same package nor subclasses
- The three access specifiers, **private**, **public**, and **protected**, provide a variety of ways to produce the many levels of access required by these categories.
- Table sums up the interactions.

### Importing Packages:

- Given that packages exist and are a good mechanism for compartmentalizing diverse classes from each other, it is easy to see why all of the built-in Java classes are stored in packages.
- There are no core Java classes in the unnamed default package; all of the standard classes are stored in some named package.

- Java includes the **import** statement to bring certain classes, or entire packages, into visibility.
- Once imported, a class can be referred to directly, using only its name. The **import** statement is a convenience to the programmer and is not technically needed to write a complete Java program.
- If you are going to refer to a few dozen classes in your application, however, the **import** statement will save a lot of typing.

This is the general form of the **import** statement:

**import *pkg1*[.*pkg2*].(*classname*|\*);**

Here, *pkg1* is the name of a top-level package, and *pkg2* is the name of a subordinate package inside the outer package separated by a dot (.).

- There is no practical limit on the depth of a package hierarchy, except that imposed by the file system.
- Finally, you specify either an explicit *classname* or a star (\*), which indicates that the Java compiler should import the entire package.
- This code fragment shows both forms in use:

```
import java.util.Date;
import java.io.*;
```

- All of the standard Java classes included with Java are stored in a package called **java**.
- The basic language functions are stored in a package inside of the **java** package called **java.lang**. This is equivalent to the following line being at the top of all of your programs:

**import java.lang.\*;**

- If a class with the same name exists in two different packages that you import using the star form, the compiler will remain silent, unless you try to use one of the classes.
- In that case, you will get a compile-time error and have to explicitly name the class specifying its package.
- It must be emphasized that the **import** statement is optional. Any place you use a class name, you can use its *fully qualified name*, which includes its full package

- For example, this fragment uses an import statement:

```
import java.util.*;  
class MyDate extends Date {  
}
```

The same example without the **import** statement looks like this:

```
class MyDate extends java.util.Date {  
}
```

In this version, **Date** is fully-qualified.

### Interfaces:

- Using the keyword **interface**, you can fully abstract a class' interface from its implementation.
- That is, using **interface**, you can specify what a class must do, but not how it does it. Interfaces are syntactically similar to classes, but they lack instance variables, and their methods are declared without any body.
- In practice, this means that you can define interfaces that don't make assumptions about how they are implemented.
- Once it is defined, any number of classes can implement an **interface**. Also, one class can implement any number of interfaces.
- To implement an interface, a class must create the complete set of methods defined by the interface. However, each class is free to determine the details of its own implementation.
- By providing the **interface** keyword, Java allows you to fully utilize the “one interface, multiple methods” aspect of polymorphism.
- Interfaces are designed to support dynamic method resolution at run time.
- Normally, in order for a method to be called from one class to another, both classes need to be present at compile time so the Java compiler can check to ensure that the method signatures are compatible.

### Defining an Interface:

- An interface is defined much like a class.
- This is the general form of an interface:

```

access interface name {
    return-type method-name1(parameter-list);
    return-type method-name2(parameter-list);
    type final-varname1 = value;
    type final-varname2 = value;
    // ...
    return-type method-nameN(parameter-list);
    type final-varnameN = value;
}

```

- When no access specifier is included, then default access results, and the interface is only available to other members of the package in which it is declared.
- When it is declared as **public**, the interface can be used by any other code.
- In this case, the interface must be the only public interface declared in the file, and the file must have the same name as the interface.
- *name* is the name of the interface, and can be any valid identifier.
- Notice that the methods that are declared have no bodies. They end with a semicolon after the parameter list.
- They are, essentially, abstract methods; there can be no default implementation of any method specified within an interface.
- Each class that includes an interface must implement all of the methods.
- Variables can be declared inside of interface declarations.
- They are implicitly **final** and **static**, meaning they cannot be changed by the implementing class. They must also be initialized.
- All methods and variables are implicitly **public**.

Here is an example of an interface definition. It declares a simple interface that contains one method called **callback()** that takes a single integer parameter.

```

interface Callback {
    void callback(int param);
}

```

### Implementing Interfaces:

Once an interface has been defined, one or more classes can implement that interface. 22

- To implement an interface, include the **implements** clause in a class definition, and then create the methods defined by the interface.
- The general form of a class that includes the **implements** clause looks like this:

```
class classname [extends superclass] [implements interface [,interface...]] {
    // class-body
}
```

- The methods that implement an interface must be declared **public**.
- Also, the type signature of the implementing method must match exactly the type signature specified in the **interface** definition

Here is a small example class that implements the **Callback** interface shown earlier.

```
class Client implements Callback {
    // Implement Callback's interface
    public void callback(int p) {
        System.out.println("callback called with " + p);
    }
}
```

- Notice that **callback( )** is declared using the **public** access specifier.
- It is both permissible and common for classes that implement interfaces to define additional members of their own.
- For example, the following version of **Client** implements **callback( )** and adds the method **nonInterfaceMeth( )**:

```
class Client implements Callback {
    // Implement Callback's interface
    public void callback(int p) {
        System.out.println("callback called with " + p);
    }
    void nonInterfaceMeth() {
        System.out.println("Classes that implement interfaces " +
            "may also define other members, too.");
    }
}
```

**Accessing Implementations Through Interface References:**

- You can declare variables as object references that use an interface rather than a class type.
- Any instance of any class that implements the declared interface can be referred to by such a variable.
- When you call a method through one of these references, the correct version will be called based on the actual instance of the interface being referred to. This is one of the key features of interfaces
- The method to be executed is looked up dynamically at run time, allowing classes to be created later than the code which calls methods on them.
- The calling code can dispatch through an interface without having to know anything about the “callee.”
- The following example calls the **callback( )** method via an interface reference variable:

```
class TestIface {
    public static void main(String args[]) {
        Callback c = new Client();
        c.callback(42);
    }
}
```

The output of this program is shown here:

callback called with 42

- While the preceding example shows, mechanically, how an interface reference variable can access an implementation object, it does not demonstrate the polymorphic power of such a reference.
- To sample this usage, first create the second implementation of **Callback**, shown here:

```
// Another implementation of Callback.
class AnotherClient implements Callback {
    // Implement Callback's interface
    public void callback(int p) {
        System.out.println("Another version of callback");
        System.out.println("p squared is " + (p*p));
    }
}
```



```

    }
}

```

Now, try the following class:

```

class TestIface2 {
    public static void main(String args[]) {
        Callback c = new Client();
        AnotherClient ob = new AnotherClient();
        c.callback(42);
        c = ob; // c now refers to AnotherClient object
        c.callback(42);
    }
}

```

The output from this program is shown here:

callback called with 42

Another version of callback

p squared is 1764

### Partial Implementations:

- If a class includes an interface but does not fully implement the methods defined by that interface, then that class must be declared as **abstract**.
- For example:

```

abstract class Incomplete implements Callback {
    int a, b;
    void show() {
        System.out.println(a + " " + b);
    }
    // ...
}

```

- Here, the class **Incomplete** does not implement **callback( )** and must be declared as **abstract**.
- Any class that inherits **Incomplete** must implement **callback( )** or be declared **abstract** itself.

**Nested Interfaces:**

- An interface can be declared a member of a class or another interface. Such an interface is called a *member interface* or a *nested interface*.
- A nested interface can be declared as **public**, **private**, or **protected**.
- This differs from a top-level interface, which must either be declared as **public** or use the default access level, as previously described.
- When a nested interface is used outside of its enclosing scope, it must be qualified by the name of the class or interface of which it is a member.
- Thus, outside of the class or interface in which a nested interface is declared, its name must be fully qualified.

Here is an example that demonstrates a nested interface:

```
class A {  
    // this is a nested interface  
    public interface NestedIF {  
        boolean isNotNegative(int x);  
    }  
}  
  
// B implements the nested interface.  
class B implements A.NestedIF {  
    public boolean isNotNegative(int x) {  
        return x < 0 ? false : true;  
    }  
}  
  
class NestedIFDemo {  
    public static void main(String args[]) {  
        // use a nested interface reference  
        A.NestedIF nif = new B();  
        if(nif.isNotNegative(10))  
            System.out.println("10 is not negative");  
        if(nif.isNotNegative(-12))  
            System.out.println("this won't be displayed");  
    }  
}
```

```
}
```

- Notice that **A** defines a member interface called **NestedIF** and that it is declared **public**.
- Next, **B** implements the nested interface by specifying implements **A.NestedIF**
- Notice that the name is fully qualified by the enclosing class' name.
- Inside the **main( )** method, an **A.NestedIF** reference called **nif** is created, and it is assigned a reference to a **B** object. Because **B** implements **A.NestedIF**, this is legal.

### Applying Interfaces:

- To understand the power of interfaces, let's look at a more practical example. In earlier chapters, you developed a class called **Stack** that implemented a simple fixed-size stack. However, there are many ways to implement a stack.
- First, here is the interface that defines an integer stack. Put this in a file called **IntStack.java**.

This interface will be used by both stack implementations.

// Define an integer stack interface.

```
interface IntStack {
    void push(int item); // store an item
    int pop(); // retrieve an item
}
```

```
class FixedStack implements IntStack {
    private int stck[];
    private int tos;
    // allocate and initialize stack
    FixedStack(int size) {
        stck = new int[size];
        tos = -1;
    }
    // Push an item onto the stack
    public void push(int item) {
        if(tos==stck.length-1) // use length member
            System.out.println("Stack is full.");
    }
}
```

```
        else
            stck[++tos] = item;
    }
    // Pop an item from the stack
    public int pop() {
        if(tos < 0) {
            System.out.println("Stack underflow.");
            return 0;
        }
        else
            return stck[tos--];
    }
}

class IFTest {
    public static void main(String args[]) {
        FixedStack mystack1 = new FixedStack(5);
        FixedStack mystack2 = new FixedStack(8);
        // push some numbers onto the stack
        for(int i=0; i<5; i++) mystack1.push(i);
        for(int i=0; i<8; i++) mystack2.push(i);
        // pop those numbers off the stack
        System.out.println("Stack in mystack1:");
        for(int i=0; i<5; i++)
            System.out.println(mystack1.pop());
        System.out.println("Stack in mystack2:");
        for(int i=0; i<8; i++)
            System.out.println(mystack2.pop());
    }
}
```

**Variables in Interfaces:**

- You can use interfaces to import shared constants into multiple classes by simply declaring an interface that contains variables that are initialized to the desired values.
- It is as if that class were importing the constant fields into the class name space as **final** variables.
- The next example uses this technique to implement an automated “decision maker”:

```
import java.util.Random;
interface SharedConstants {
    int NO = 0;
    int YES = 1;
    int MAYBE = 2;
    int LATER = 3;
    int SOON = 4;
    int NEVER = 5;
}
class Question implements SharedConstants {
    Random rand = new Random();
    int ask() {
        int prob = (int) (100 * rand.nextDouble());
        if (prob < 30)
            return NO; // 30%
        else if (prob < 60)
            return YES; // 30%
        else if (prob < 75)
            return LATER; // 15%
        else if (prob < 98)
            return SOON; // 13%
        else
            return NEVER; // 2%
    }
}
class AskMe implements SharedConstants {
    static void answer(int result) {
        Department of CSE(AI&ML), GSSSSETW, Mysuru
```

```
        switch(result) {
        case NO:
            System.out.println("No");
            break;
        case YES:
            System.out.println("Yes");
            break;
        case MAYBE:
            System.out.println("Maybe");
            break;
        case LATER:
            System.out.println("Later");
            break;
        case SOON:
            System.out.println("Soon");
            break;
        case NEVER:
            System.out.println("Never");
            break;
        }
    }
}

public static void main(String args[]) {
    Question q = new Question();
    answer(q.ask());
    answer(q.ask());
    answer(q.ask());
    answer(q.ask());
}
}
```

### Interfaces Can Be Extended:

- One interface can inherit another by use of the keyword **extends**. The syntax is the same as for inheriting classes.

- When a class implements an interface that inherits another interface, it must provide implementations for all methods defined within the interface inheritance chain. Following is an example:

```
// One interface can extend another. interface
A {
    void meth1(); void
    meth2();
}
// B now includes meth1() and meth2() -- it adds meth3(). interface B
extends A {
    void meth3();
}
// This class must implement all of A and B class
MyClass implements B {
    public void meth1() {
        System.out.println("Implement meth1().");
    }
    public void meth2() {
        System.out.println("Implement meth2().");
    }
    public void meth3() {
        System.out.println("Implement meth3().");
    }
}
class IFExtend {
    public static void main(String arg[]) { MyClass
        ob = new MyClass(); ob.meth1();
        ob.meth2();
        ob.meth3();
    }
}
```

## Exception handling:

### Introduction

An **Exception**, It can be defined as an abnormal event that occurs during program execution and disrupts the normal flow of instructions. The abnormal event can be an error in the program.

Errors in a java program are categorized into two groups:

1. **Compile-time errors** occur when you do not follow the syntax of a programming language.
2. **Run-time errors** occur during the execution of a program.

### Concepts of Exceptions

An exception is a run-time error that occurs during the execution of a java program.



Example: If you divide a number by zero or open a file that does not exist, an exception is raised.

In java, exceptions can be handled either by the java run-time system or by a userdefined code. When a run-time error occurs, an exception is thrown. The unexpected situations that may occur during program execution are:

- Running out of memory Resource
- allocation errors Inability to find files
- Problems in network connectivity

#### **Exception handling techniques:**

Java exception handling is managed via five keywords they are:

1. try:
2. catch.
3. throw.
4. throws.
5. finally.

#### **Exception handling Statement Syntax :**

Exceptions are handled using a try-catch-finally construct, which has the Syntax.

```
try
{
    <code>
}
catch (<exception type1> <parameter1>)
{
    // 0 or more<statements>
}
finally
{
    // finally block<statements>
}
```

1. **try Block:** The java code that you think may produce an exception is placed within a try block for a suitable catch block to handle the error.

If no exception occurs the execution proceeds with the finally block else it will look for the matching catch block to handle the error.

Again if the matching catch handler is not found execution proceeds with the finally block and the default exception handler throws an exception.

2. **catch Block:** Exceptions thrown during execution of the try block can be caught and handled in a catch block. On exit from a catch block, normal execution continues and the finally block is executed (Though the catch block throws an exception).

3. **finally Block:** A finally block is always executed, regardless of the cause of exit from the try block, or whether any catch block was executed. Generally finally block is used for freeing resources, cleaning up, closing connections etc.

**Example:**

The following is an array is declared with 2 elements. Then the code tries to access the 3<sup>rd</sup> element of the array which throws an exception.

```
// File Name : ExcepTest.java import
java.io.*;
public class ExcepTest
{
    public static void main(String
args[])
    {
        try
        {
            int a[] = new int[2];
            System.out.println("Access element three : " + a[3]);
        }
        catch(ArrayIndexOutOfBoundsException e)
        {
            System.out.println("Exception thrown : " + e);
        }
        System.out.println("Out of the block");
    }
}
```

This would produce following result:

```
Exception thrown: java.lang.ArrayIndexOutOfBoundsException: 3 Out of the block
```

### **Multiple catch Blocks:**

A try block can be followed by multiple catch blocks. The syntax for multiple catch blocks looks like the following:

```
try
{
    // code
}

catch(ExceptionType1 e1)
{
    //Catch block
}

catch(ExceptionType2 e2)
{
    //Catch block
}
```

```
}  
catch(ExceptionType3 e3)  
{  
    //Catch block  
}
```

The previous statements demonstrate three catch blocks, but you can have any number of them after a single try.

**Example:** Here is code segment showing how to use multiple try/catch statements.

```
class Multi_Catch  
{  
    public static void main (String args  
[])  
    {  
  
        try  
        {  
            int a=args.length;  
            System.out.println("a="+a);  
int b=50/a; int  
c[]={1}  
        }  
        catch (ArithmeticException e)  
        {  
            System.out.println ("Division by zero");  
        }  
  
        catch (ArrayIndexOutOfBoundsException e)  
        {  
            System.out.println (" array index out of bound");  
        }  
    }  
}
```

**OUTPUT:**  
Division by zero array index out of  
bound

### *Nested try Statements*

Just like the multiple catch blocks, we can also have multiple try blocks. These try blocks may be written independently or we can nest the try blocks within each other, i.e., keep one try-catch block within another try-block.

The program structure for nested try statement is:

#### Syntax

```
try
{
    // statements
    // statements

    try
    {
        // statements
        // statements
    }
    catch (<exception_two> obj)
    {
        // statements
    }

    // statements
    // statements
}
catch (<exception_two> obj)
{
    // statements
}
```

Consider the following example in which you are accepting two numbers from the command line. After that, the command line arguments, which are in the string format, are converted to integers.

If the numbers were not received properly in a number format, then during the conversion a `NumberFormatException` is raised otherwise the control goes to the next try block. Inside this second try-catch block the first number is divided by the second number, and during the calculation if there is any arithmetic error, it is caught by the inner catch block.

**Example**

```
class Nested_Try
{
    public static void main (String args [ ] )
    {
        try
        {
            int a = Integer.parseInt (args [0]); int
            b = Integer.parseInt (args [1]); int
            quot = 0;

            try
            {
                quot = a / b; System.out.println(quot);
            }
            catch (ArithmeticException e)
            {
                System.out.println("divide by zero");
            }
        }
        catch (NumberFormatException e)
        {
            System.out.println ("Incorrect argument type");
        }
    }
}
```

**The output of the program is: If the arguments are entered properly in the command prompt like:**

**OUTPUT:**

```
java Nested_Try 2 4 6
4
```

If the argument contains a string than the number:

**OUTPUT**

```
java Nested_Try 2 4 aa
Incorrect argument type
```

If the second argument is entered zero:

**OUTPUT**

```
java Nested_Try 2 4 0 divide by  
zero
```

**throw Keyword:**

**throw keyword** is used to throw an exception explicitly. Only object of Throwable class or its sub classes can be thrown.

Program execution stops on encountering **throw** statement, and the closest catch statement is checked for matching type of exception.

**Syntax :** `throw ThrowableInstance`

---

### Creating Instance of Throwable class

There are two possible ways to get an instance of class Throwable,

1. Using a parameter in catch block.
2. Creating instance with **new** operator.

```
new NullPointerException("test");
```

This constructs an instance of NullPointerException with name test.

---

### Example demonstrating throw Keyword class

Test

```
{    static void avg()  
    {  
        try  
        {  
            throw new ArithmeticException("demo");  
        }  
        catch(ArithmeticException e)  
        {  
            System.out.println("Exception caught");  
        }  
    }  
}
```

```

        }
    }
    public static void main(String args[])
    {
        avg();
    }
}

```

In the above example the avg() method throw an instance of ArithmeticException, which is successfully handled using the catch statement.

---

### throws Keyword

Any method capable of causing exceptions must list all the exceptions possible during its execution, so that anyone calling that method gets a prior knowledge about which exceptions to handle. A method can do so by using the **throws** keyword.

**Syntax :**     *type method\_name(parameter\_list) throws*  
*exception\_list*  
 {  
                     //definition of method  
 }

**NOTE :** It is necessary for all exceptions, except the exceptions of type **Error** and **RuntimeException**, or any of their subclass.

---

### Example demonstrating throws Keyword class

Test

```

{
    static void check() throws ArithmeticException
    {
        System.out.println("Inside check function");
        new ArithmeticException("demo");
    }
    public static void main(String args[])
    {
        try
        {
            throw

```



```
        check();
    }
    catch(ArithmeticException e)
    {
        System.out.println("caught" + e);
    }
}
```

### finally

The finally clause is written with the try-catch statement. It is guaranteed to be executed after a catch block or before the method quits.

### Syntax

```
try
{
    // statements
}

catch (<exception> obj)
{
    // statements
} finally
{
    //statements
}
```

Take a look at the following example which has a catch and a finally block. The catch block catches the ArithmeticException which occurs for arithmetic error like divide-by-zero. After executing the catch block the finally is also executed and you get the output for both the blocks.

**Example:**

```
class Finally_Block
{
    static void division ( )
    {
        try
        {
            int
num = 34, den = 0; int
quot = num / den;
        }
        catch(ArithmeticException e)
        {
            System.out.println ("Divide by zero");
        }
        finally
        {
            System.out.println ("In the finally block");
        }
    }
}

class Mypgm
{
    public static void main(String args[])
    {
        Finally_Block f=new Finally_Block(); f.division();
    }
}
```

**OUTPUT**

Divide by zero  
In the finally block

## Java's Built in Exceptions

Java defines several exception classes inside the standard package **java.lang**.

The most general of these exceptions are subclasses of the standard type **RuntimeException**. Since **java.lang** is implicitly imported into all Java programs, most exceptions derived from **RuntimeException** are automatically available.

Java defines several other types of exceptions that relate to its various class libraries. Following is the list of Java **Unchecked RuntimeException**.

| Exception                      | Description                                             |
|--------------------------------|---------------------------------------------------------|
|                                |                                                         |
| ArithmeticException            | Arithmetic error, such as divide-by-zero.               |
| ArrayIndexOutOfBoundsException | Array index is out-of-bounds.                           |
|                                |                                                         |
| ArrayStoreException            | Assignment to an array element of an incompatible type. |
| ClassCastException             | Invalid cast.                                           |
|                                |                                                         |

|                              |                                                                   |
|------------------------------|-------------------------------------------------------------------|
| IllegalArgumentException     | Illegal argument used to invoke a method.                         |
| IllegalMonitorStateException | Illegal monitor operation, such as waiting on an unlocked thread. |
| IllegalStateException        | Environment or application is in incorrect state.                 |
| IllegalThreadStateException  | Requested operation not compatible with current thread state.     |
| IndexOutOfBoundsException    | Some type of index is out-of-bounds.                              |
| NegativeArraySizeException   | Array created with a negative size.                               |
| NullPointerException         | Invalid use of a null reference.                                  |
| NumberFormatException        | Invalid conversion of a string to a numeric format.               |

|                                 |                                                  |
|---------------------------------|--------------------------------------------------|
| SecurityException               | Attempt to violate security.                     |
| StringIndexOutOfBoundsException | Attempt to index outside the bounds of a string. |
| UnsupportedOperationException   | An unsupported operation was encountered.        |

Following is the list of **Java Checked Exceptions** Defined in java.lang.

| Exception                  | Description                                                                 |
|----------------------------|-----------------------------------------------------------------------------|
| ClassNotFoundException     | Class not found.                                                            |
| CloneNotSupportedException | Attempt to clone an object that does not implement the Cloneable interface. |
| IllegalAccessException     | Access to a class is denied.                                                |
| InstantiationException     | Attempt to create an object of an abstract class or interface.              |
| InterruptedException       | One thread has been interrupted by another thread.                          |
| NoSuchFieldException       | A requested field does not exist.                                           |

|                       |                                    |
|-----------------------|------------------------------------|
| NoSuchMethodException | A requested method does not exist. |
|-----------------------|------------------------------------|

### Creating your own Exception Subclasses

Here you can also define your own exception classes by extending **Exception**. These exception can represents specific runtime condition of course you will have to throw them yourself, but once thrown they will behave just like ordinary exceptions.

When you define your own exception classes, choose the ancestor carefully. Most custom exception will be part of the official design and thus checked, meaning that they extend Exception but not RuntimeException.

#### Example: Throwing User defined Exception

```
public class MyException extends Exception
{
    String msg = ""; int
marks=50;
    public MyException()
    {
    }
    public MyException(String str)
    {
super(str);
    }
    public String
    {
        if(marks <= 40)
msg = "You have failed";
if(marks > 40)
    msg = "You have Passed";

    }
}
return msg;
```

```
class test
{
    public static void main(String args[])
    {
        test t = new test();
        t.dd();
    }

    public void add()
    {
        try
        {
            int i=0;
            if( i<40)
                throw new MyException();
        }
        catch(MyException ee1)
        {
            System.out.println("Result:"+ee1);
        }
    }
}
```

**OUTPUT**

Result: You have Passed

### Chained Exception

Chained exceptions are the exceptions which occur one after another i.e. most of the time to response to an exception are given by an application by throwing another exception.

Whenever in a program the first exception causes an another exception, that is termed as **Chained Exception**. Java provides new functionality for chaining exceptions.

Exception chaining (also known as "nesting exception") is a technique for handling the exception, which occur one after another i.e. most of the time



is given by an application to response to an exception by throwing another exception.

Typically the second exception is caused by the first exception. Therefore chained exceptions help the programmer to know when one exception causes another.

The **constructors** that support chained exceptions in **Throwable** class are:

Throwable

initCause(Throwable)

Throwable(Throwable)

Throwable(String,  
Throwable) Throwable  
getCause()